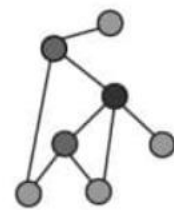
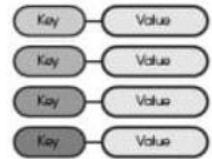
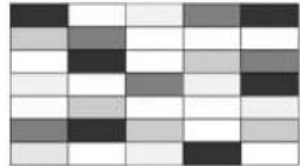
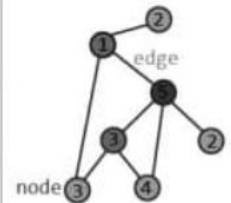
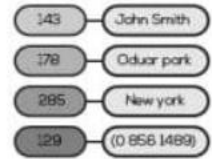


NoSQL Databases / MongoDB.

What is NoSQL

- NoSQL Database is used to refer a non-SQL or non relational database.
- It provides a mechanism for storage and retrieval of data other than tabular relations model used in relational databases. NoSQL database doesn't use tables for storing data. It is generally used to store big data and real-time web applications.

Different kinds of NOSQL databases.

Document	Graph	Key-Value	Wide-Column													
																
<pre>{ "user": { "id": "143", "name": "improgrammer", "city": "New York" } }</pre>			<table><tr><td>1</td><td>Fruit</td><td>A Foo</td><td>B Baz</td></tr><tr><td>2</td><td>City</td><td>E DC</td><td>D PLA</td><td>G FLD</td></tr><tr><td>3</td><td>State</td><td>A NZ</td><td>C CL</td></tr></table>	1	Fruit	A Foo	B Baz	2	City	E DC	D PLA	G FLD	3	State	A NZ	C CL
1	Fruit	A Foo	B Baz													
2	City	E DC	D PLA	G FLD												
3	State	A NZ	C CL													
																

NOSQL databases.

- **Document Databases** – These Db usually pair each key with a complex data structure which is called a document. Documents can contain key-array pairs or key-value pairs or even nested documents. Examples of document NoSQL: MongoDB, Apache CouchDB, Raven DB, ArangoDB, Couchbase, Cosmos DB, IBM Domino, MarkLogic, OrientDB.
- **Key-value stores** – Every single item is stored as a Key-value pair. Key-value stores are the most *simple* database among all NoSQL Databases. Examples of Key-value NoSQL – Redis, Memcached, Apache Ignite, Riak.
- **Wide-column stores** – These types of Databases are optimized for queries over large datasets, and instead of rows, they store columns of data together. Examples of Wide column NoSQL – Cassandra, Hbase, Scylla.
- **Graph stores** – These store information about graphs, networks, such as social connections, road maps, transport links. Examples of Graph NoSQL – Neo4j, AllegroGraph.

SQL vs NoSQL

	SQL	NoSQL
Database Type	Relational Databases	Non-relational Databases / Distributed Databases
Structure	Table-based	• Key-value pairs • Document-based • Graph databases • Wide-column stores
Scalability	Designed for scaling up vertically by upgrading one expensive custom-built hardware	Designed for scaling out horizontally by using shards to distribute load across multiple commodity (inexpensive) hardware
Strength	• Great for highly structured data and don't anticipate changes to the database structure • Working with complex queries and reports	• Pairs well with fast paced, agile development teams • Data consistency and integrity is not top priority • Expecting high transaction load

Installing and configuring MongoDB

- Download mongo db from the below given link and can be installed.
- https://www.mongodb.com/try/download/community?tck=docs_server

Install Mongo Shell from the below given URL:

<https://www.mongodb.com/try/download/shell>

Mongo DB Data types

Data Types	Description
String	String is the most commonly used datatype. It is used to store data. A string must be UTF 8 valid in mongodb.
Integer	Integer is used to store the numeric value. It can be 32 bit or 64 bit depending on the server you are using.
Boolean	This datatype is used to store boolean values. It just shows YES/NO values.
Double	Double datatype stores floating point values.
Min/Max Keys	This datatype compare a value against the lowest and highest bson elements.
Arrays	This datatype is used to store a list or multiple values into a single key.
Object	Object datatype is used for embedded documents.
Null	It is used to store null values.
Symbol	It is generally used for languages that use a specific type.
Date	This datatype stores the current date or time in unix time format. It makes you possible to specify your own date time by creating object of date and pass the value of date, month, year into it.

Start MondoDB Server

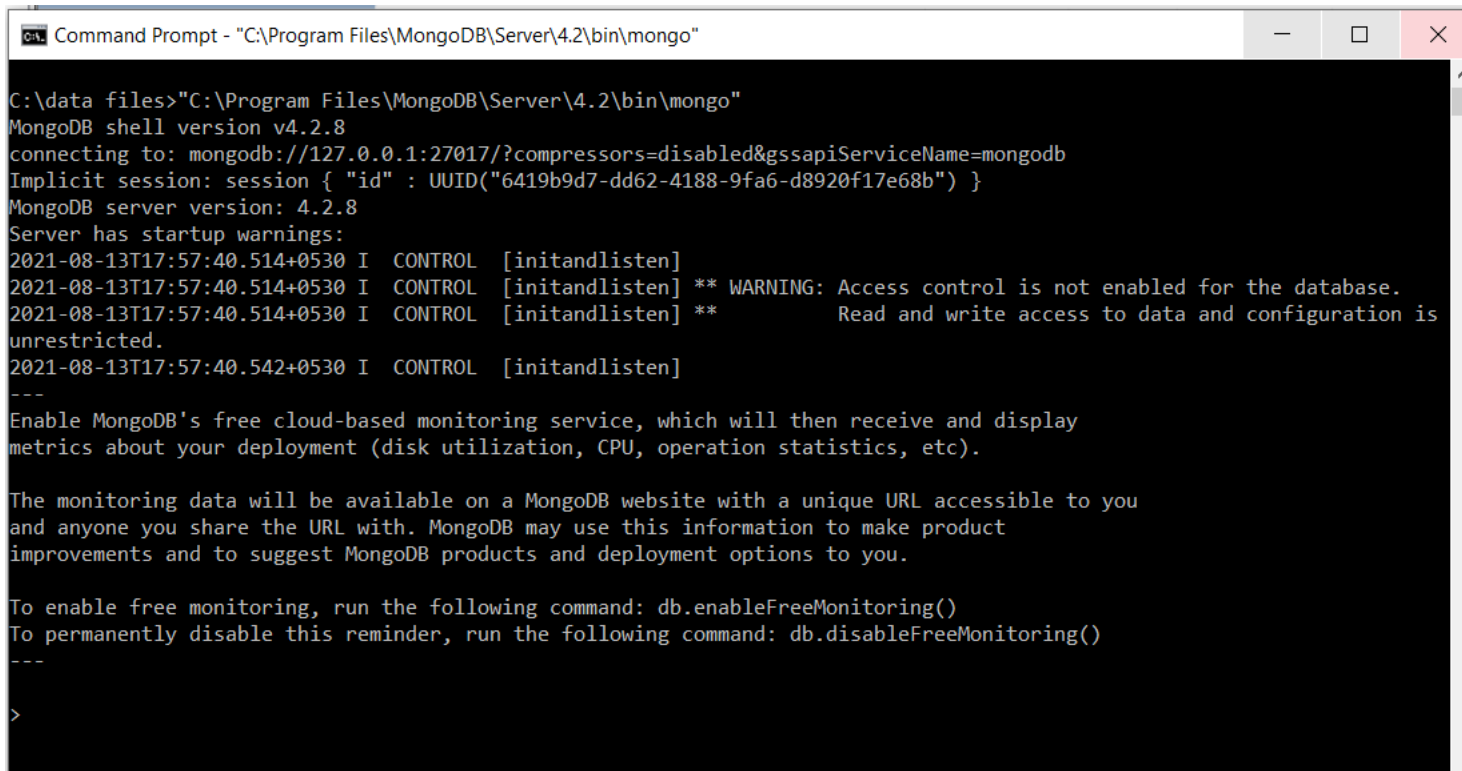
- From the command prompt give mongod.
This is Mongo DB Server.

```
Command Prompt - mongod
Directory of C:\Program Files\MongoDB\Server\4.2\bin
11-06-2020 16:35      15,182,288 bsondump.exe
11-06-2020 17:02      21,469,696 mongo.exe
11-06-2020 17:06      36,290,048 mongod.exe
11-06-2020 16:35      23,347,783 mongodump.exe
11-06-2020 16:35      22,974,076 mongoexport.exe
11-06-2020 16:35      22,906,257 mongofiles.exe
11-06-2020 16:35      23,232,501 mongoimport.exe
11-06-2020 16:35      23,844,627 mongorestore.exe
11-06-2020 17:05      18,511,872 mongos.exe
11-06-2020 16:35      22,518,351 mongostat.exe
11-06-2020 16:35      22,077,487 mongotop.exe
          11 File(s)      252,354,986 bytes
           0 Dir(s)      157,961,138 bytes free

C:\Program Files\MongoDB\Server\4.2\bin>mongod
2021-08-13T17:58:51.947+0530 I  CONTROL  [main] Automatically disabling TLS 1.0, to force-enable TLS 1.0 specify --sslDisabledProtocols 'none'
2021-08-13T17:58:51.951+0530 W  ASIO      [main] No TransportLayer configured during NetworkInterface startup
2021-08-13T17:58:51.953+0530 I  CONTROL  [initandlisten] MongoDB starting : pid=12436 port=27017 dbpath=C:\data\db\ 64-bit host=LAPTOP-ONEMN5DS
2021-08-13T17:58:51.953+0530 I  CONTROL  [initandlisten] targetMinOS: Windows 7/Windows Server 2008 R2
2021-08-13T17:58:51.954+0530 I  CONTROL  [initandlisten] db version v4.2.8
2021-08-13T17:58:51.954+0530 I  CONTROL  [initandlisten] git version: 43d25964249164d76d5e04dd6cf38f6111e21f5f
2021-08-13T17:58:51.954+0530 I  CONTROL  [initandlisten] allocator: tcmalloc
2021-08-13T17:58:51.954+0530 I  CONTROL  [initandlisten] modules: none
2021-08-13T17:58:51.954+0530 I  CONTROL  [initandlisten] build environment:
2021-08-13T17:58:51.954+0530 I  CONTROL  [initandlisten]     distmod: 2012plus
2021-08-13T17:58:51.954+0530 I  CONTROL  [initandlisten]     distarch: x86_64
2021-08-13T17:58:51.954+0530 I  CONTROL  [initandlisten]     target_arch: x86_64
2021-08-13T17:58:51.954+0530 I  CONTROL  [initandlisten] options: {}
2021-08-13T17:58:52.016+0530 I  STORAGE  [initandlisten] Detected data files in C:\data\db\ created by the 'wiredTiger' storage engine, so setting the activ
```


Start mongo client or Mongo Shell.

- Go to any directory where data needs to be maintained and give mongo.exe as shown in the screen below:

A screenshot of a Windows Command Prompt window. The title bar reads "Command Prompt - \"C:\\Program Files\\MongoDB\\Server\\4.2\\bin\\mongo\"". The command prompt shows the execution of "C:\\data files>\"C:\\Program Files\\MongoDB\\Server\\4.2\\bin\\mongo\"". The output shows the MongoDB shell version v4.2.8, connection details to localhost:27017, and startup warnings. A warning states: "** WARNING: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted." The prompt then displays information about MongoDB's free cloud-based monitoring service and provides commands to enable or disable it.

```
C:\data files>"C:\Program Files\MongoDB\Server\4.2\bin\mongo"
MongoDB shell version v4.2.8
connecting to: mongodb://127.0.0.1:27017/?compressors=disabled&gssapiServiceName=mongodb
Implicit session: session { "id" : UUID("6419b9d7-dd62-4188-9fa6-d8920f17e68b") }
MongoDB server version: 4.2.8
Server has startup warnings:
2021-08-13T17:57:40.514+0530 I  CONTROL  [initandlisten]
2021-08-13T17:57:40.514+0530 I  CONTROL  [initandlisten] ** WARNING: Access control is not enabled for the database.
2021-08-13T17:57:40.514+0530 I  CONTROL  [initandlisten] **           Read and write access to data and configuration is
unrestricted.
2021-08-13T17:57:40.542+0530 I  CONTROL  [initandlisten]
---
Enable MongoDB's free cloud-based monitoring service, which will then receive and display
metrics about your deployment (disk utilization, CPU, operation statistics, etc).

The monitoring data will be available on a MongoDB website with a unique URL accessible to you
and anyone you share the URL with. MongoDB may use this information to make product
improvements and to suggest MongoDB products and deployment options to you.

To enable free monitoring, run the following command: db.enableFreeMonitoring()
To permanently disable this reminder, run the following command: db.disableFreeMonitoring()
---
```

Create a database

- Use <database>
- If the database exists then this command opens the existing database and uses it. If the given database does not exist then it creates the database given in <database> and opens it and uses it.
- “db” command in shell will show which database mongo db is using right now.
- “show dbs” command will display all the databases. Immediately after creating the database, it is now shown as it does not contain any Table/collections. You can create a collection / table by using “insert” command.
- Db.movie.insert ({"name:"Vikings"})

Use database and db command

```
Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\r
> use MyDB
switched to db MyDB
>
>
> db
MyDB
>
> show dbs
admin    0.000GB
config  0.000GB
local    0.000GB
test     0.000GB
> db.movie.insert ({"name":"Vikings"})
WriteResult({ "nInserted" : 1 })
> show dbs
MyDB     0.000GB
admin    0.000GB
config  0.000GB
local    0.000GB
test     0.000GB
>
```

Drop a database.

- `Db.dropDatabase()` will drop the currently selected database.

```
Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

> show dbs
MyDB      0.000GB
admin     0.000GB
config    0.000GB
local     0.000GB
test      0.000GB
>
>
> use MyDB
switched to db MyDB
>
> db.dropDatabase()
{ "dropped" : "MyDB", "ok" : 1 }
>
> show dbs
admin     0.000GB
config    0.000GB
local     0.000GB
test      0.000GB
>
```

Create a collection.

- Collection in mongodb is equivalent to table name in SQL. There are two ways to create a collection
 - (1) `db.createCollection (<name of collection>)`
 - (2) Write directly into the collection, if the collection does not exist then it will create the collection and then will write into it. By using the command `db.<collection name>.insert ({“attribute name”: Value, “attribute name”:value})`.
- Collection is Table in SQL and document is a record in SQL.
- “`db.collection_name.find()`” will display all the documents in the collection or in SQL → will display all the records of the table.

Create collection and inserting data into it.

C:\> Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

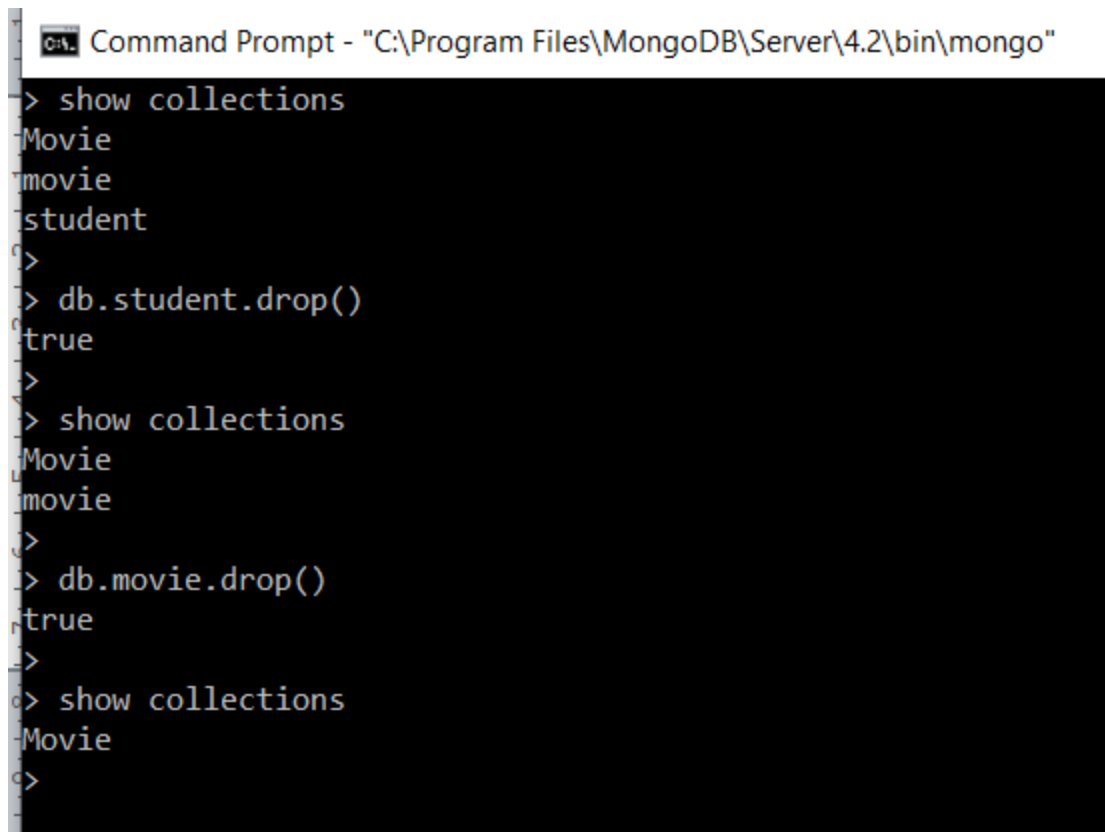
```
> db.createCollection ("Movie")
{ "ok" : 1 }
> show collections
Movie
>
> db.movie.insert ({"name":"vikings"})
WriteResult({ "nInserted" : 1 })
>
> db.movie.find()
{ "_id" : ObjectId("6116b16b0358355939723a1c"), "name" : "vikings" }
>
> db.movie.insert ({
... "name":"Money Heist","Rating":4})
WriteResult({ "nInserted" : 1 })
>
> db.movie.find()
{ "_id" : ObjectId("6116b16b0358355939723a1c"), "name" : "vikings" }
{ "_id" : ObjectId("6116b1a10358355939723a1d"), "name" : "Money Heist", "Rating" : 4 }
>
```

Creating collection using insert command.

```
Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"
> db.student.insert ({ "id":200,"name":"Surya","stream":"CS","location":"Bangalore"})
WriteResult({ "nInserted" : 1 })
>
> show collections
Movie
movie
student
>
>
> db.student.find()
{ "_id" : ObjectId("6116b3000358355939723a20"), "id" : 200, "name" : "Surya", "stream" : "CS", "location" : "Bangalore"
}
```

Drop the collection.

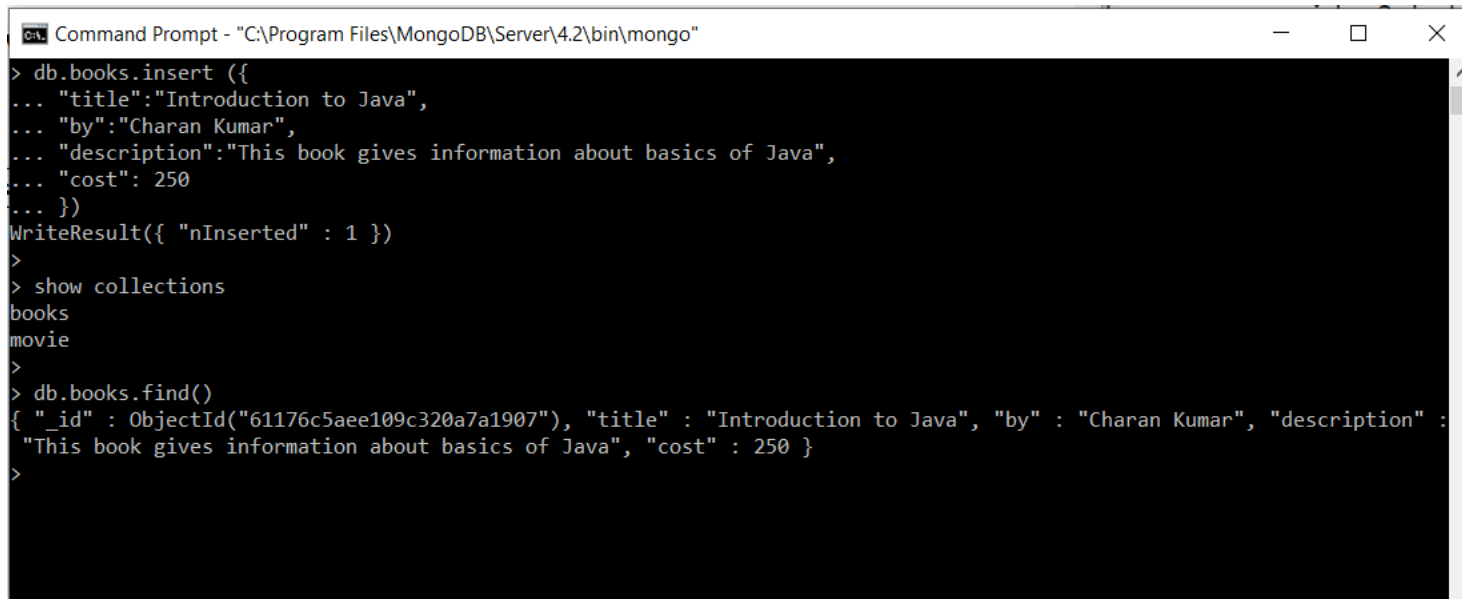
- Collection / Table name in sql can be dropped by the command.
`db.<collection name>.drop()`



```
Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"
> show collections
Movie
movie
student
>
> db.student.drop()
true
>
> show collections
Movie
movie
>
> db.movie.drop()
true
>
> show collections
Movie
>
```


Insert a document

- Document is a Record in SQL. Db.<collection Name>.insert will insert a document into a collection as show in the image below:



```
Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"
> db.books.insert ({
...  "title":"Introduction to Java",
...  "by":"Charan Kumar",
...  "description":"This book gives information about basics of Java",
...  "cost": 250
... })
WriteResult({ "nInserted" : 1 })
>
> show collections
books
movie
>
> db.books.find()
{ "_id" : ObjectId("61176c5aee109c320a7a1907"), "title" : "Introduction to Java", "by" : "Charan Kumar", "description" :
"This book gives information about basics of Java", "cost" : 250 }
>
```

- Db.books.insert() will add a document into a collection. Db.books.find() will display all the documents in the collection.
- Writing the documents in a notepad and copy paste in the shell is a better option.

Inserting multiple documents in a collection.

*Untitled - Notepad

File Edit Format View Help

```
db.books.insert ([
{
title:"Introduction to Spring",
by:"Srinath",
description:"This is about Spring",
cost: 350
},
{
title:"Angular",
by:"Simi Dwivedi",
description:"Insight of Angular is given in this book",
cost: 350
},
{
title:"Explore React",
by:"Amisha Sharma",
description:"Explore the topics of React in depth",
cost: 350
}
```

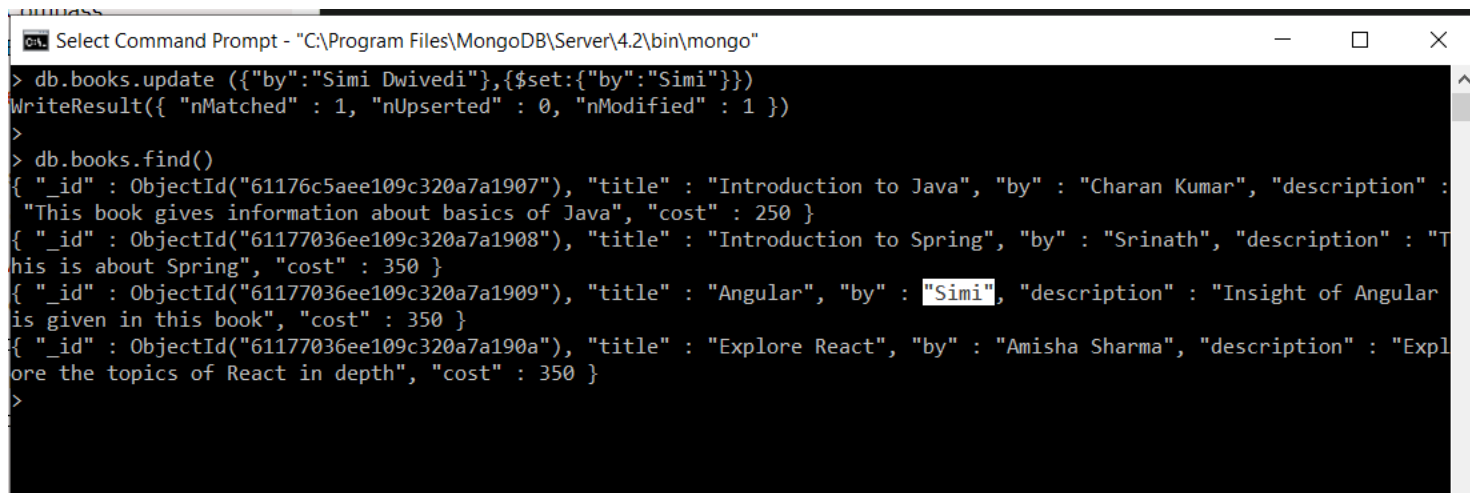
Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

```
> db.books.insert ([
... {
...   title:"Introduction to Spring",
...   by:"Srinath",
...   description:"This is about Spring",
...   cost: 350
... },
... {
...   title:"Angular",
...   by:"Simi Dwivedi",
...   description:"Insight of Angular is given in this book",
...   cost: 350
... },
... {
...   title:"Explore React",
...   by:"Amisha Sharma",
...   description:"Explore the topics of React in depth",
...   cost: 350
... }
... ]);
BulkWriteResult({
  "writeErrors" : [ ],
  "writeConcernErrors" : [ ],
  "nInserted" : 3,
  "nUpserted" : 0,
  "nMatched" : 0,
  "nModified" : 0,
  "nRemoved" : 0,
  "upserted" : [ ]
})
>
```

```
> db.books.find()
{ "_id" : ObjectId("61176c5aee109c320a7a1907"), "title" : "Introduction to Java", "by" : "Charan Kumar", "description" : "This book gives information about basics of Java", "cost" : 250 }
{ "_id" : ObjectId("61177036ee109c320a7a1908"), "title" : "Introduction to Spring", "by" : "Srinath", "description" : "This is about Spring", "cost" : 350 }
{ "_id" : ObjectId("61177036ee109c320a7a1909"), "title" : "Angular", "by" : "Simi Dwivedi", "description" : "Insight of Angular is given in this book", "cost" : 350 }
{ "_id" : ObjectId("61177036ee109c320a7a190a"), "title" : "Explore React", "by" : "Amisha Sharma", "description" : "Explore the topics of React in depth", "cost" : 350 }
>
```

Document update.

- A document can be updated using `db.<collection name>.update()` command.
- Using `db.books.update()` Author Simi Dwivedi is modified to Simi.



```
Select Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"
> db.books.update ({"by":"Simi Dwivedi"},{$set:{"by":"Simi"}})
WriteResult({ "nMatched" : 1, "nUpserted" : 0, "nModified" : 1 })
>
> db.books.find()
{ "_id" : ObjectId("61176c5aee109c320a7a1907"), "title" : "Introduction to Java", "by" : "Charan Kumar", "description" :
  "This book gives information about basics of Java", "cost" : 250 }
{ "_id" : ObjectId("61177036ee109c320a7a1908"), "title" : "Introduction to Spring", "by" : "Srinath", "description" : "T
his is about Spring", "cost" : 350 }
{ "_id" : ObjectId("61177036ee109c320a7a1909"), "title" : "Angular", "by" : "Simi", "description" : "Insight of Angular
is given in this book", "cost" : 350 }
{ "_id" : ObjectId("61177036ee109c320a7a190a"), "title" : "Explore React", "by" : "Amisha Sharma", "description" : "Expl
ore the topics of React in depth", "cost" : 350 }
>
```

Removing a document

- `Db.<collection name>.remove()` is used to remove or delete documents from the collection. A criteria can be specified.

Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

```
> db.books.remove ({"title":"Angular"})
WriteResult({ "nRemoved" : 1 })
>
> db.books.find()
{ "_id" : ObjectId("61176c5aee109c320a7a1907"), "title" : "Introduction to Java", "by" : "Charan Kumar", "description" : "This book gives information about basics of Java", "cost" : 250 }
{ "_id" : ObjectId("61177036ee109c320a7a1908"), "title" : "Introduction to Spring", "by" : "Srinath", "description" : "This is about Spring", "cost" : 350 }
{ "_id" : ObjectId("61177036ee109c320a7a190a"), "title" : "Explore React", "by" : "Amisha Sharma", "description" : "Explore the topics of React in depth", "cost" : 350 }
```

- Here the document titled Angular is removed.

Removing all the documents.

- `Db.<Collection name>.remove({})` will remove all the documents from the collection.

Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

```
> db.books.remove({})
WriteResult({ "nRemoved" : 3 })
>
> db.books.find()
>
>
```

Querying documents.

- `Db.collection name.find()` → Will retrieve all the documents.
- `Db.collection name.find (<condition>)` → Will retrieve all the documents based on the given condition.

Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

```
> db.books.find()
{ "_id" : ObjectId("6117944bee109c320a7a190d"), "title" : "Introduction to Spring", "author" : "Charan", "description" : "Introduction to Spring", "cost" : 350 }
{ "_id" : ObjectId("6117944bee109c320a7a190e"), "title" : "React", "author" : "Anu", "description" : "React", "cost" : 350 }
{ "_id" : ObjectId("6117944bee109c320a7a190f"), "title" : "Angular", "author" : "Johnson", "description" : "Angular", "cost" : 350 }
{ "_id" : ObjectId("6117950dee109c320a7a1910"), "title" : "Introduction to Java", "author" : "Simi", "description" : "Introduction to java", "cost" : 350 }
>
>
>
>
> db.books.find({"author":"Simi"})
{ "_id" : ObjectId("6117950dee109c320a7a1910"), "title" : "Introduction to Java", "author" : "Simi", "description" : "Introduction to java", "cost" : 350 }
>
```

Querying and displaying using pretty()

- Db.<collection name>.find().pretty() will display the data/documents in a pretty manner.

Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

```
> db.books.find().pretty()
{
  "_id" : ObjectId("6117944bee109c320a7a190d"),
  "title" : "Introduction to Spring",
  "author" : "Charan",
  "description" : "Introduction to Spring",
  "cost" : 350
}
{
  "_id" : ObjectId("6117944bee109c320a7a190e"),
  "title" : "React",
  "author" : "Anu",
  "description" : "React",
  "cost" : 350
}
{
  "_id" : ObjectId("6117944bee109c320a7a190f"),
  "title" : "Angular",
  "author" : "Johnson",
  "description" : "Angular",
  "cost" : 350
}
{
  "_id" : ObjectId("6117950dee109c320a7a1910"),
  "title" : "Introduction to Java",
  "author" : "Simi",
  "description" : "Introduction to java",
  "cost" : 350
}
>
```


FindOne()

- db.collection name.findOne() → will display the first record.

C:\> Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

```
> db.books.findOne()
{
  "_id" : ObjectId("6117944bee109c320a7a190d"),
  "title" : "Introduction to Spring",
  "author" : "Charan",
  "description" : "Introduction to Spring",
  "cost" : 350
}
```

Adding conditions in find()

Operation	Syntax	Example	RDBMS Equivalent
Equality	{<key>:{\$eq:<value>}}	db.mycol.find({"by":"tutorials point"}).pretty()	where by = 'tutorials point'
Less Than	{<key>:{\$lt:<value>}}	db.mycol.find({"likes":{\$lt:50}}).pretty()	where likes < 50
Less Than Equals	{<key>:{\$lte:<value>}}	db.mycol.find({"likes":{\$lte:50}}).pretty()	where likes <= 50
Greater Than	{<key>:{\$gt:<value>}}	db.mycol.find({"likes":{\$gt:50}}).pretty()	where likes > 50
Greater Than Equals	{<key>:{\$gte:<value>}}	db.mycol.find({"likes":{\$gte:50}}).pretty()	where likes >= 50
Not Equals	{<key>:{\$ne:<value>}}	db.mycol.find({"likes":{\$ne:50}}).pretty()	where likes != 50

- Using conditions in find().

Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

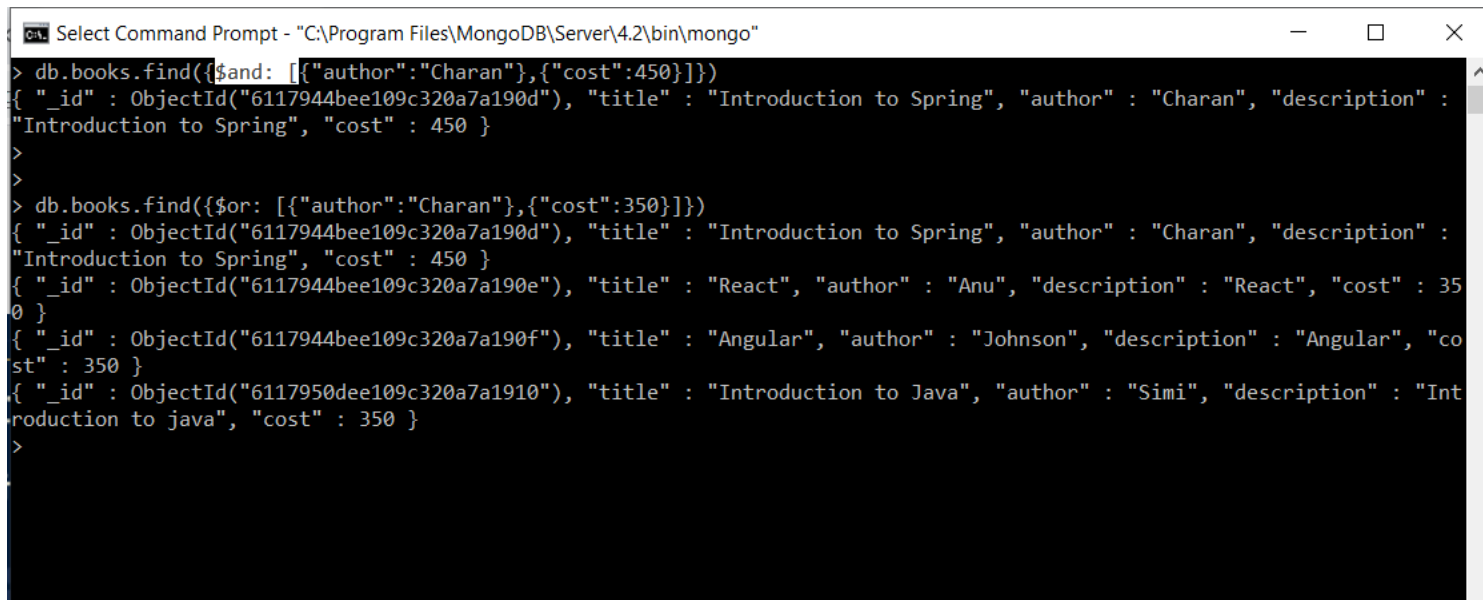
```
> db.books.find({cost:450}).pretty()
{
  "_id" : ObjectId("6117944bee109c320a7a190d"),
  "title" : "Introduction to Spring",
  "author" : "Charan",
  "description" : "Introduction to Spring",
  "cost" : 450
}

> db.books.find({cost:{$gt:350}}).pretty()
{
  "_id" : ObjectId("6117944bee109c320a7a190d"),
  "title" : "Introduction to Spring",
  "author" : "Charan",
  "description" : "Introduction to Spring",
  "cost" : 450
}

> db.books.find().pretty()
{
  "_id" : ObjectId("6117944bee109c320a7a190d"),
  "title" : "Introduction to Spring",
  "author" : "Charan",
  "description" : "Introduction to Spring",
  "cost" : 450
}
{
  "_id" : ObjectId("6117944bee109c320a7a190e"),
  "title" : "React",
  "author" : "Anu",
  "description" : "React",
  "cost" : 350
}
{
  "_id" : ObjectId("6117944bee109c320a7a190f"),
  "title" : "Angular",
  "author" : "Johnson",
  "description" : "Angular",
  "cost" : 350
}
{
  "_id" : ObjectId("6117950dee109c320a7a1910"),
  "title" : "Introduction to Java",
  "author" : "Simi",
  "description" : "Introduction to java",
  "cost" : 350
}
>
```

Adding and an or to find().

- `Db.collection name.find({$and: [{attribute:value}, {attribute:value}]})` → Gets the document anding two conditions and displays the result. The same is with `or`. For `or` `$or` is to be used.

A screenshot of a Windows Command Prompt window titled "Select Command Prompt - 'C:\Program Files\MongoDB\Server\4.2\bin\mongo'". The window shows two MongoDB queries and their results. The first query uses the \$and operator to find documents where the author is 'Charan' and the cost is 450. The second query uses the \$or operator to find documents where either the author is 'Charan' and the cost is 350, or the author is 'Anu' and the cost is 350, or the author is 'Johnson' and the cost is 350, or the author is 'Simi' and the cost is 350.

```
> db.books.find({$and: [{"author":"Charan"}, {"cost":450}]})
{ "_id" : ObjectId("6117944bee109c320a7a190d"), "title" : "Introduction to Spring", "author" : "Charan", "description" : "Introduction to Spring", "cost" : 450 }
>
>
> db.books.find({$or: [{"author":"Charan"}, {"cost":350}]})
{ "_id" : ObjectId("6117944bee109c320a7a190d"), "title" : "Introduction to Spring", "author" : "Charan", "description" : "Introduction to Spring", "cost" : 450 }
{ "_id" : ObjectId("6117944bee109c320a7a190e"), "title" : "React", "author" : "Anu", "description" : "React", "cost" : 350 }
{ "_id" : ObjectId("6117944bee109c320a7a190f"), "title" : "Angular", "author" : "Johnson", "description" : "Angular", "cost" : 350 }
{ "_id" : ObjectId("6117950dee109c320a7a1910"), "title" : "Introduction to Java", "author" : "Simi", "description" : "Introduction to java", "cost" : 350 }
>
```

- First condition, author should be charan and cost should be 450.
- Second condition, with `or`, author is charan and cost is 350. Results are displayed based on the given condition.

Projection.

- Projection is displaying the fields we want and not displaying the fields which we do not want.
- `Db.collection name.find({}, {key:0/1})` → when the key is set to 1 that field is displayed else it will not be displayed.

Limit() and Skip() in find().

- Limit(n) will display first n records from the collection.
- Skip(n) will skip n records from the beginning.
- Syntax : db.collection name.find().limit (n) or db.collection name.find().skip(2).limit(1)

```
Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

db.books.find()
{ "_id" : ObjectId("6117944bee109c320a7a190d"), "title" : "Introduction to Spring", "author" : "Charan", "description" : "Introduction to Spring", "cost" : 450 }
{ "_id" : ObjectId("6117944bee109c320a7a190e"), "title" : "React", "author" : "Anu", "description" : "React", "cost" : 350 }
{ "_id" : ObjectId("6117944bee109c320a7a190f"), "title" : "Angular", "author" : "Johnson", "description" : "Angular", "cost" : 350 }
{ "_id" : ObjectId("6117950dee109c320a7a1910"), "title" : "Introduction to Java", "author" : "Simi", "description" : "Introduction to java", "cost" : 350 }
>
>
> db.books.find().limit(2)
{ "_id" : ObjectId("6117944bee109c320a7a190d"), "title" : "Introduction to Spring", "author" : "Charan", "description" : "Introduction to Spring", "cost" : 450 }
{ "_id" : ObjectId("6117944bee109c320a7a190e"), "title" : "React", "author" : "Anu", "description" : "React", "cost" : 350 }
>
>
> db.books.find().skip(2).limit(1)
{ "_id" : ObjectId("6117944bee109c320a7a190f"), "title" : "Angular", "author" : "Johnson", "description" : "Angular", "cost" : 350 }
>
```

Sorting the data.

- Sort() is used to sort the data displayed from collection.
- Syntax is sort ({key/field name:1/-1}) → 1 sorts the field in ascending order and -1 sorts in descending order.
- Db.books.find().sort ({title:1}) → Displays all the records after sorting field title in the ascending order and next command author sorts in descending order.

Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

```
db.books.find()
{"_id" : ObjectId("6117944bee109c320a7a190d"), "title" : "Introduction to Spring", "author" : "Charan", "description" : "Introduction to Spring", "cost" : 450 }
{"_id" : ObjectId("6117944bee109c320a7a190e"), "title" : "React", "author" : "Anu", "description" : "React", "cost" : 350 }
{"_id" : ObjectId("6117944bee109c320a7a190f"), "title" : "Angular", "author" : "Johnson", "description" : "Angular", "cost" : 350 }
{"_id" : ObjectId("6117950dee109c320a7a1910"), "title" : "Introduction to Java", "author" : "Simi", "description" : "Introduction to java", "cost" : 350 }

db.books.find().sort({title:1})
{"_id" : ObjectId("6117944bee109c320a7a190f"), "title" : "Angular", "author" : "Johnson", "description" : "Angular", "cost" : 350 }
{"_id" : ObjectId("6117950dee109c320a7a1910"), "title" : "Introduction to Java", "author" : "Simi", "description" : "Introduction to java", "cost" : 350 }
{"_id" : ObjectId("6117944bee109c320a7a190d"), "title" : "Introduction to Spring", "author" : "Charan", "description" : "Introduction to Spring", "cost" : 450 }
{"_id" : ObjectId("6117944bee109c320a7a190e"), "title" : "React", "author" : "Anu", "description" : "React", "cost" : 350 }

db.books.find().sort({author:-1})
{"_id" : ObjectId("6117950dee109c320a7a1910"), "title" : "Introduction to Java", "author" : "Simi", "description" : "Introduction to java", "cost" : 350 }
{"_id" : ObjectId("6117944bee109c320a7a190f"), "title" : "Angular", "author" : "Johnson", "description" : "Angular", "cost" : 350 }
{"_id" : ObjectId("6117944bee109c320a7a190d"), "title" : "Introduction to Spring", "author" : "Charan", "description" : "Introduction to Spring", "cost" : 450 }
{"_id" : ObjectId("6117944bee109c320a7a190e"), "title" : "React", "author" : "Anu", "description" : "React", "cost" : 350 }
```

Indexing.

- Indexing on a particular field will improve performance while retrieving data on that particular field.
- Indexing in MongoDB can be created using `db.collectionName.createIndex({field name:1/-1})` → 1 will index the field/key in ascending order, -1 will index the field in descending order.
- `Db.collectionName.getIndexes()` → will display all the available indexes of the collection.

Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

```
> db.books.createIndex ({"title":1})
{
  "createdCollectionAutomatically" : false,
  "numIndexesBefore" : 1,
  "numIndexesAfter" : 2,
  "ok" : 1
}
>
> db.books.getIndexes()
[
  {
    "v" : 2,
    "key" : {
      "_id" : 1
    },
    "name" : "_id_",
    "ns" : "MyDB.books"
  },
  {
    "v" : 2,
    "key" : {
      "title" : 1
    },
    "name" : "title_1",
    "ns" : "MyDB.books"
  }
]
```


Remove the Index.

- dropIndex() is used to remove the Index.
- Syntax is : db.collection name.dropIndex {(Key:1/-1)}.
-

Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

```
> db.books.dropIndex ({"title":1})
{ "nIndexesWas" : 2, "ok" : 1 }
>
> db.books.getIndexes()
[
  {
    "v" : 2,
    "key" : {
      "_id" : 1
    },
    "name" : "_id_",
    "ns" : "MyDB.books"
  }
]
>
```

Aggregate Functions

- Some of aggregate functions are Min, Max, Avg, Sum.
- Syntax is :
- `db.books.aggregate([{$group: {_id:null, MinimumCost:{$min:"$cost"}}}])`

It groups by Id, since it is null all the records are considered individually. MinimumCost is the string to be displayed after performing aggregating function. `$min/$max/$avg/$sum` can be used as aggregate functions. And the next is the field or key on which aggregate functions to be operated upon.

Aggregate functions

```
Command Prompt - "C:\Program Files\MongoDB\Server\4.2\bin\mongo"

> db.books.find()
{ "_id" : ObjectId("6117944bee109c320a7a190d"), "title" : "Introduction to Spring", "author" : "Charan", "description" : "Introduction to Spring", "cost" : 450 }
{ "_id" : ObjectId("6117944bee109c320a7a190e"), "title" : "React", "author" : "Anu", "description" : "React", "cost" : 350 }
{ "_id" : ObjectId("6117944bee109c320a7a190f"), "title" : "Angular", "author" : "Johnson", "description" : "Angular", "cost" : 350 }
{ "_id" : ObjectId("6117950dee109c320a7a1910"), "title" : "Introduction to Java", "author" : "Simi", "description" : "Introduction to java", "cost" : 350 }
>
>
>
> db.books.aggregate([{$group: {_id:null,"TotalCount":{$sum:1}}}])
{ "_id" : null, "TotalCount" : 4 }
>
>
> db.books.aggregate([{$group: {_id:"$by_user",Total_Cost:{$sum:"$cost"}}}])
{ "_id" : null, "Total_Cost" : 1500 }
>
>
> db.books.aggregate([{$group: {_id:null,Max_cost:{$max:"$cost"}}}])
{ "_id" : null, "Max_cost" : 450 }
>
>
> db.books.aggregate([{$group: {_id:null,Min_Cost:{$min:"$cost"}}}])
{ "_id" : null, "Min_Cost" : 350 }
>
>
> db.books.aggregate([{$group: {_id:null, Average_Cost:{$avg:"$cost"}}}])
{ "_id" : null, "Average_Cost" : 375 }
>
>
```

Thank you!